CarTogether – Išmanioji bendrų kelionių ir išlaidų dalijimosi sistema

Kursinio darbo I dalis: projektavimo dokumentas

Pildymo instrukcija: pakeiskite tekstą laužtiniuose skliaustuose savo atsakymais. Lentelėse pridėkite tiek eilučių, kiek reikia. Prieš pateikdami pašalinkite šią instrukciją ir kitus pildymo paaiškinimus. Rekomenduojama apimtis – 3–5 puslapiai arba panašios apimties Markdown dokumentas.

Šiame dokumente nenurodykite vardo, pavardės, grupės ar studento numerio. Dokumentą įkelkite į projekto „GitHub“ repozitoriją. Repozitorijos nuorodą ir studento numerį pateikite užduoties apraše nurodytoje „Google“ formoje. Pagal formoje pateiktus duomenis repozitorija bus susieta su studentu.

1. Problema ir idėja

Sistema vienu sakiniu: Sistema skirta žmonėms, keliaujantiems tarp miestų arba mieste, automatiškai rasti tinkamus bendrakeleivius, suderinti maršrutus ir teisingai pasidalinti kelionės kuro išlaidas.

Problema ir dabartinis procesas: Dabar vairuotojai, keliaujantys vieni, patiria dideles kuro išlaidas. Norintieji rasti bendrakeleivių naudojasi „Facebook“ grupėmis ar tradicinėmis platformomis („BlaBlaCar“, „BananaCar“), kur reikia atlikti daug rankinio paieškos darbo, žinutes rašyti atskirai, o maršrutai ir laikai dažnai tiksliai nesutampa.

Nauda: Suvienodinus ir automatizavus procesą, keleiviai sutaupo laiko ieškodami kelionių, o vairuotojai sumažina kelionės išlaidas be papildomo streso derinant maršrutus.

Naudotojai:

Vairuotojai: skelbia planuojamas keliones, nurodo laisvas vietas ir kuro sąnaudas.

Keleiviai: iekško kelionių pagal išvykimo/atvykimo vietas bei laiką, rezervuoja veitas ir vertina vairuotojus.

Prielaidos: Darome prielaidą, kad vairuotojai sąžiningai nurodo automobilio kuro sąnaudas ir laikosi suderinto paėmimo laiko, o kelionės vyksta Lietuvos maršrutais.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Pagrindinis modulis ar pagalbinė funkcija
Automatinis kelionų suderinimas	Įvedus maršrutą, sistema atrenka ir suderina geriausiai atitinkančias keliones pagal laiko langą ir maršruto nuokrypį	Pagrindinis modulis
Kelionės skelbimas ir valdymas	Vairuotojas gali sukurti kelionę, nurodyti išvykimo/atvykimo vietas, laiką, laisvų vietų skaičių ir automobilio duomenis	Pagalbinė funkcija
Rezervaijos patvirtinimas	Keleivis gali išsiųsti užklausą ir gauti patvirtinimą apie rezervuotą vietą automobilyje	Pagalbinė funkcija
Naudotojo profilis ir įvertinimai	Naudotojai gali peržiūrėti kitų profilius, atsiliepimus bei palikti įvertinimą po kelionės	Pagalbinė funkcija

Į kursinio darbo apimtį neįeina: Integruotas tiesioginis mokėjimų apdorojimas (mokesčiai atsiskaitomi grynaisiais arba pavedimu tarp naudotojų), gyvas žemėlapių sekimas (real-time GPS tracking).

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Automatinio kelionių ir bendrakeleivių suderinimo. Modulis atsakingas už tinkamiausių kelionių paiešką pagal keleivio užklausą, maršrutų nuokrypio skaičiavimą bei kuro išlaidų padalinimą.

Logika, kurią reikės projektuoti ir testuoti: Maršruto geografinio atstumo skaičiavimo patikra, laiko lango atitikimo logika, maksimalios leistinos kainos vienam keleiviui ribojimas ir vietų blokavimas išvengiant dvigubo rezervavimo.

Įvestis: „Vilnius (Saulėtekis)“, atvykimo taškas „Kaunas (Centras)“, data/laikas „2026-10-10 15:00“, leistinas laiko nuokrypis ±30 min.

Išvestis: Išrikiuotas geriausiai atitinkančių vairuotojų sąrašas su apskaičiuota kuro dalies kaina (pvz., Vairuotojas Titas, paėmimas 15:15, kaina: 4.50 €).

Veikimo eiga:

1. Kelevis pateikia išvykimo, atvykimo vietas ir norimą laiką.
2. Modulis atrenka aktyvias keliones ta pačia kryptimi pateiktame laiko lange.
3. Skaičiuojama, ar keleivio paėmimo/išlaipinimo taškai neviršija leistino vairuotojo maršruto nuokrypio.
4. Apskaičiuojama rekomenduojama kuro kaina vienam asmeniui.

Taisyklės arba sprendimo žingsniai

1. Maršruto nuokrypio taisyklė: Papildomas atstumas, kurį vairuotojas turi nuvažiuoti norėdamas paimti ir išlaipinti keleivį, negali padidinti bendro kelionės laiko daugiau nei 15 minučių.

2. Sąžiningos kainos taisyklės: Maksimali kelionės kaina keleiviui skaičiuojama pagal formulę:

$$Kaina = \frac{Bendros\ kuro\ išlaidos}{Laisvų\ vietų\ skaičius + 1}$$

Užkertamas kelias fiksuoti komercines, pelno siekiančias kainas

3. Prieinamumo ir reitingo taisyklė: Kelionų paieškoje neatvaizduojami vairuotojai, kurių vidutinis įvertinimas yra žemesnis nei 3.0 žvaigždutės arba kurių laisvų vietų skaičius yra 0.

Scenarijai būsimiems testams

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Įprastas atvejis	Yra aktyvi vairuotojo kelionė Vilnius-Kaunas (3 laisvos vietos, išvykimas 15:00). Keleivis ieško kelionės Vilnius-Kaunas 15:10	Keleivis vykdo paiešką	Pasiūloma ši kelionė, apskaičiuota kaina - 4.50 €, būsena „Galima rezervuoti“
Ribinis atvejis arba konfliktas	Automobilyje likusi 1 laisva vieta. Du keleiviai vienu metu (vidutiniškai 50 ms skirtumu) paspaudžia „Rezervuoti“	Sistemai pateikiamos dvi užklausos iš karto	Pirmajam keleiviui rezervacija patvirtinama, antrajam – atmetama su pranešimu „Laisvų vietų nebėra“
Klaida arba neįmanomas rezultatas	Keleivis ieško kelionės maršrutu Panevėžys–Šiauliai, tačiau atitinkamą dieną joks vairuotojas nevažiuoja	Vykdoma paieška	Grąžinamas tuščias sąrašas su pranešimu „Kelionių nerasta“ ir siūlymu prenumeruoti pranešimus

Jei modulis naudoja AI: Netaikoma

4. Kokybės atributas

Pasirinktas atributas: Patikimumas.

Kodėl svarbus šiai sistemai: Populiariais maršrutais (pvz., penktadienio popietę Vilnius–Palanga) keli keleiviai gali vienu metu bandyti rezervuoti paskutinę vietą automobilyje. Būtina užtikrinti, kad ta pati vieta nebus rezervuota du kartus.

Tikrinimo scenarijus ir sąlygos: Lokalioje aplinkoje imuliuojama 50 vienalaikių užklausų, kurios bando rezervuoti paskutinę likusią vietą toje pačioje kelionėje.

Sėkmės kriterijus: Tiksliai 1 užklausa gauna patvirtinimą, o likusios 49 užklausos gauna kalidos pranešimą. Nėra duomenų bazės neatitikimų.

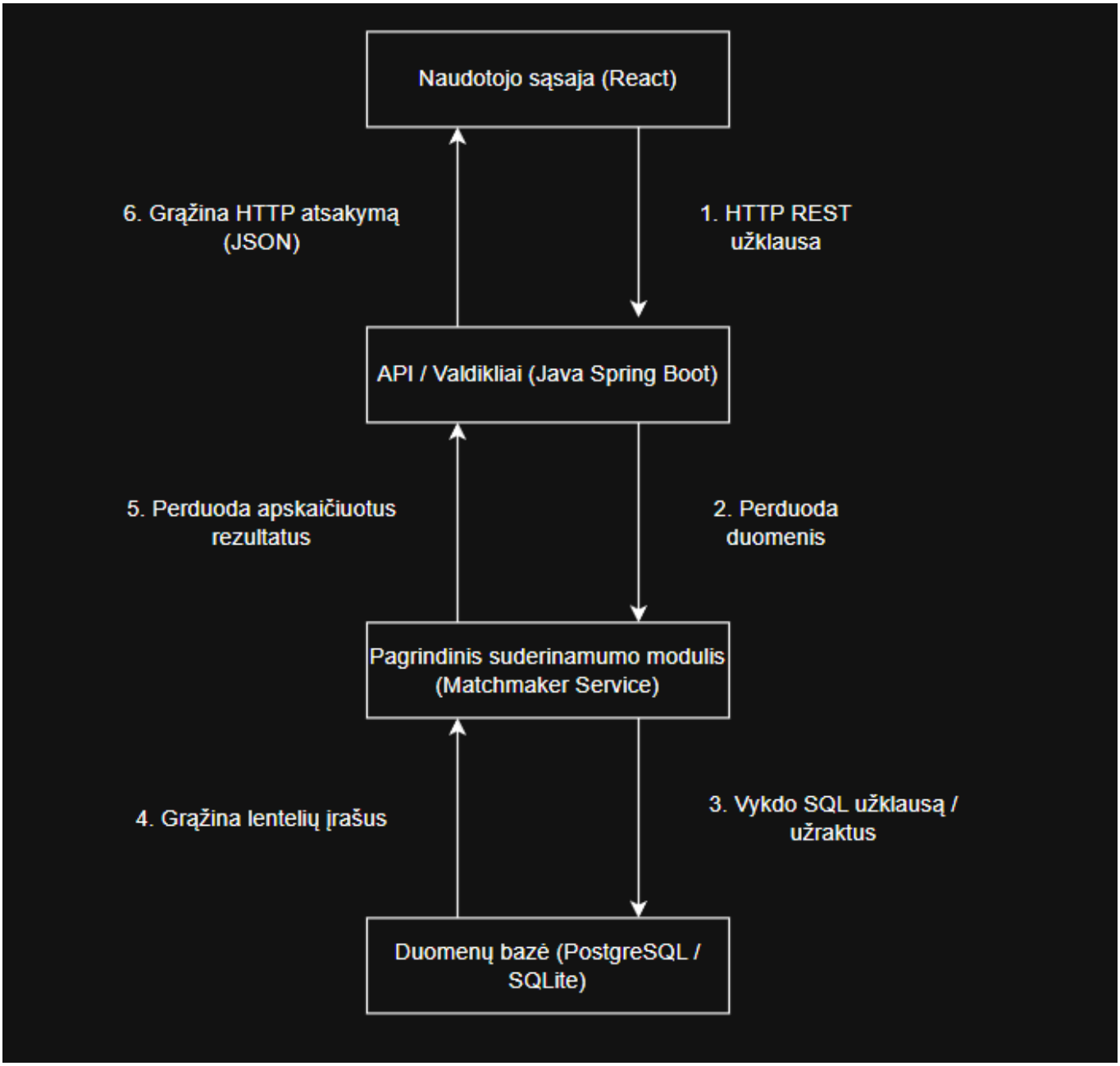
Numatytas projektavimo sprendimas: Apsauga nuo dvigubo rezervavimo: užtikrinama naudojant duomenų bazės užraktus. Kai pirmasis naudotojas pradeda rezervaciją, sistema trumpam užblokuoja pasirinktą vietą automobilyje, kad kitas naudotojas, tuo pat metu atliekantis veiksmą, matytų atnaujintą (užimtą) būseną ir negautų tos pačios vietos.

Kaip patikrinsiu vėlesniame etape: Atliekant automatizuotą integracinį / apkrovos testą naudojant įrankį.

Sprendimo kaina arba ribojimas: Mikrosekundėmis prailgsta rezervacijos apdorojimo laikas dėl užrakto taikymo duomenų bazėje.

5. Pradinė sistemos struktūra

Paprasta schema



Sistemos dalis	Atsakomybė
Naudotojo sąsaja (React)	Duomenų įvedimas (kelionės paieškos forma, kelionės skelbimo forma) ir gautų rezultatų atvaizdavimas naudotojui.
API / Valdikliai (Java Spring Boot)	Priima HTTP užklausas iš sąsajos, atlieka naudotojo autorizaciją, tikrina įvesties duomenis ir nukreipia juos į verslo logikos sluoksnį.
Pagrindinis suderinimo modulis (Matchmaker Service)	Vykdo kelionių atrankos, laiko lango patikros, maršruto nuokrypio ir kuro kainos skaičiavimo taisykles.
Duomenų bazė (PostgreSQL / SQLite)	Saugomi naudotojų profiliai, paskelbtų kelionių duomenys, rezervacijų būsenos ir atsiliepimai.

Planuojamos technologijos ir pasirinkimo priežastys:

Java / Spring Boot (Backend): Pasirinkta dėl griežtos struktūros, aukšto patikimumo ir plataus paplitimo pramonėje. „Spring Boot“ karkasas leidžia patogiai atskirti verslo logikos sluoksnį nuo API valdiklių, o su „Spring Data JPA“ itin paprasta valdyti duomenų bazės transakcijas ir užraktus (Optimistic / Pessimistic Locking), užtikrinant apsaugą nuo dvigubo vietų rezervavimo.

PostgreSQL (Pagrindinė duomenų bazė): Patikima relacinė duomenų bazė, užtikrinanti tvarias transakcijas (ACID) bei eilutės lygio užrakinimą, kas apsaugo nuo dvigubo rezervavimo konfliktų. Be to, ji turi palaikymą geografiniams duomenims (PostGIS), kas leidžia patogiai skaičiuoti atstumus ir maršrutų nuokrypius.

SQLite / H2 (Kūrimo ir testavimo duomenų bazė): Kūrimo bei automatizuoto testavimo (JUnit) etape naudojama lengva, įterptinė duomenų bazė, nereikalaujanti atskiro serverio konfigūravimo ir leidžianti greitai patikrinti scenarijus lokalioje aplinkoje.

React (Frontend): Pasirinkta reaktyviai, greitai ir patogiai naudotojo sąsajai sukurti, kuri bendrauja su Java backend'u per REST API sąsają.

6. AI panaudojimas

AI rengiant šį dokumentą

[Nurodykite, ar naudojote AI. Jei taip, užpildykite lentelę.]

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
Gemini (Koncepto ir dokumentacijos projektavimas)	Naudojau sistemos architektūros, testavimo scenarijų ir taisyklių suformulavimo pavyzdžiams gauti	Atmečiau pernelyg sudėtingas AI maršrutizavimo funkcijas, nes jos viršija kursinio darbo apimtį.	Patikrinau, ar pateikti scenarijai ir taisyklės atitinka užduoties aprašo reikalavimus.

Planuojamas AI naudojimas kuriant sistemą

Kur ir kam naudosiu AI: AI priemones (pvz., GitHub Copilot / ChatGPT / Gemini) planuoju naudoti kaip pagalbinį įrankį kodo rašymui: bazinėms CRUD operacijoms, DTO objektų generavimui ir automatizuotų vieneto (Unit) testų ruošiniams.

Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą: Kiekvienas sugeneruotas kodo fragmentas bus peržiūrimas rankiniu būdu, tikrinama, ar jis atitinka projektavimo taisykles, ir paleidžiami automatiniai testai veikimui patvirtinti.

Ar AI bus sistemos funkcionalumo dalis: Ne.

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
1. DB ir API projektavimas	Duomenų bazės schema ir REST API sąrašas	1. Suprojektuoti lenteles 2. Aprašyti REST galinius taškus ir DTO objektus
2. Backend kodo rašymas	Veikianti Java Spring Boot sistema su testais	1. Parašyti kelionų paieškos logiką. 2. Įdėti DB užraktus nuo dvigubo rezervavimo
3. Frontbend ir sujungimas	Veikiantis React prototipas	1. Sukurti paieškos ir skelbimo formas. 2. Sujungti React sąsają su Java backend‘u 3. Patikrinti, ar veikia rezervacija

Būsimo prototipo veikimo scenarijus: Vairuotojas paskelbia kelionę Vilnius-Kaunas su 3 laisvomis vietomis. Keleivis atlieka paiešką, mato apskaičiuotą kainą 4.50 € ir paspaudžia „Rezervuoti“. Sistema sėkmingai patvirtina rezervaciją ir sumažina laisvų vietų skaičių iki 2.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Maršruto nuokrypio skaičiavimo sudėtingumas	Pradžioje skaičiuoti atstumus supaprastintai – pagal mieste esančius rajonus ar fiksuotus taškus.
Dvigubo vietų rezervavimo rizika	Parašyti automatinį testą, kuris imituos kelias rezervacijas vienu metu, ir panaudoti DB užraktus.